Scythe Apertus: Sovereign Autonomous Software Repair

Hack Apertus 2026 · Online · Track 2B: Own Project
Team: Syrava
Project: An open-source, on-premises repair agent using Apertus v1.5
Demo: See `docs/DEMO_SCRIPT.md`; live recording link must be entered on the submission portal.
Code: See the public repository containing `track_2b/`.

1. Abstract and problem

AI-generated software patches can be persuasive yet incorrect. Developers also need assurances about source-code custody, tool permissions and whether a model's asserted success matches an independently observed test result. Scythe Apertus is a deliberately constrained, open-source prototype: Apertus v1.5 diagnoses a small defect, writes a complete replacement for one allowlisted file and optionally retries after a failing test. A separate, offline Docker verifier runs independent assertions and returns observed results. The application records per-attempt evidence, SHA-256 hashes, model identity and an explicit pass/fail decision. The project includes twelve originally authored software defects, a web demonstration and a Docker `make run` launch path. The broader Scythe/BlackScythe IDE predates this event; the independently packaged Apertus prototype is the Hack Apertus output.

2. Architecture and sovereign deployment

Target architecture (mandatory): on-premises. The controller, offline verifier and OpenAI-compatible Apertus model server can run on equipment administered by the using organization. The model is separately served from locally held Apertus weights, e.g. through vLLM. In this configuration source code need not leave the operator's network. A public hosted endpoint is possible for development, but **is not required by, and is not evidence of, a sovereign deployment**. Actual local model serving was not reproduced in the authoring environment.

A separate `compose.onprem.yaml` overlay supplies the Swiss AI Initiative's official vLLM model-server image for NVIDIA GPU hosts, completing the optional fully local deployment topology. This GPU configuration was not executed on the authoring machine. The controller serves an accessible local UI at `127.0.0.1:8787` and accepts a benchmark case ID. It sends the issue and broken source to Apertus via `LLM_NAME`, `LLM_BASE_URL` and optional `LLM_API_KEY`. After validating the JSON patch schema, it writes the one allowed replacement (`bug.cjs`) to a Docker request volume. The verifier, which has **no network and receives no model credentials**, reads the request using a read-only mount, launches a bounded Node test process as a non-root user, and writes pass/fail output to a separate volume. The controller reads that response and may request one additional Apertus attempt. The final result requires both the independent test pass and a passing second-model critique.

Flow: Issue + failing fixture → Apertus diagnosis → allowlisted patch → offline verifier → bounded retry → Apertus critique → SHA-256 evidence JSON.

Two Docker Compose services use a read-only image filesystem, disabled privilege escalation and Linux capability drops. The verifier also uses `network_mode: none`, PID/memory/CPU limits, and a deliberately scrubbed subprocess environment. The app has no worker-execution endpoint other than selecting the twelve curated cases. Docker is defense in depth, not a formal containment proof; the worker process and the child code still share an isolated filesystem namespace.

3. Purposeful Apertus integration

The configured model is `swiss-ai/Apertus-v1.5-8B` by default. Apertus is responsible for **(a)** concise root-cause diagnosis, **(b)** a minimal complete-file repair proposal, **(c)** a bounded correction using the previous verifier's error, and **(d)** a final evidence-based critique. These calls use the OpenAI-compatible `/chat/completions` schema. The prompts require JSON with explicit keys; patches outside `bug.cjs`, empty outputs and oversized source files are rejected before reaching the verifier. Model messages

do not include reference fixes or hidden test source. Only the isolated verifier observes actual test outcomes; model confidence alone never determines success.

This architecture uses no proprietary model-specific features. The model endpoint can be swapped for an operator-managed installation of Apertus v1.5 8B/70B without changing the repair controller. The benchmark executes on Node rather than on the LLM GPU. **No model weights, private repositories, secrets or third-party copyrighted excerpts are bundled.**

4. Dataset and evaluation methodology

`data/cases.json` contains twelve original CommonJS defect/repair pairs covering: pagination boundaries; allowlisted hostname suffixes; exponential backoff caps; falsy configuration values; stable case-insensitive deduplication; exact-record lookup; numeric clamping; inclusive date windows; cache expiration; numeric sort; zero integer parsing; and post-discount tax computation. Every record includes a source statement, per-item license, an issue description, broken source, reference solution, hidden Node assertions, and a personal-data flag (false). The dataset is licensed CDLA-Permissive-2.0; example code is also Apache-2.0. No human-subject data was collected.

Metric: A model repair is resolved only when the offline verifier's tests pass **and** Apertus's final critique says `pass`. Evidence includes the original and patch SHA-256, each verifier outcome/hash, model identifier, timestamps, token totals if supplied, and a final record digest. We deliberately report model failures, exceptions and timeouts rather than deleting them from the denominator. These tiny examples are *not* SWE-bench, and public tests permit possible contamination or reward gaming.

Check	Observed result on October 15, 2026
Broken original fixtures fail independent Node assertions	**12 / 12**
Reference fixes pass those assertions	**12 / 12**
Local automated Node test suites (includes mocked LLM integration)	**19 / 19**
Genuine Apertus model repair attempts	**Not run**
Real Apertus repair rate / token cost / latency	**Not measured**
Docker Compose launch on a Docker-enabled host	**Not executed in authoring host**

The integration test uses an explicitly mocked language-model endpoint to exercise HTTP → controller → file queue → verifier → evidence. It proves the wiring under test conditions; **it is not evidence that Apertus made the repair**. Docker Compose YAML was independently parsed and checked for an offline verifier; the Docker CLI was not present in the authoring environment.

5. Reproducibility

Run `cd track_2b && make run` on a Docker-enabled host. Open `http://localhost:8787` and select **Verify fixture** to reproduce an offline original-fails/reference-passes check. This requires no model. For a genuine Apertus repair, set `LLM_NAME` to the served Apertus model identifier, `LLM_BASE_URL` to its on-premises OpenAI-compatible endpoint and `LLM_API_KEY` only if needed. Restart `make run`, select a case and click **Run Apertus repair**. Retrieve the evidence JSON from the persistent Docker `evidence` volume. On a machine with Node.js 20+, `node --test tests/*.test.mjs` runs the independent package regression suite without GitHub Actions, CI providers, or model API costs.

To reproduce reported actual model performance, record the model weight revision, serving stack, hardware, endpoint configuration, exact source commit, prompts, all failure records, token usage and fixture IDs. This submission deliberately does not invent values for steps that have not yet been run.

6. Limitations, value, and next work

Scope: The released demonstration addresses twelve small synthetic single-file JavaScript problems. It does not claim real-world repo-scale fault localization, coverage of arbitrary languages, production-

ready patch signing, adversarial-proof sandboxing, or the ability to fix unconstrained bugs. The initial benchmark's test code is public; generated code may discover those tests at runtime, so a private external holdout is a future requirement. The UI has no multi-user authentication and binds to loopback via Docker; exposing it publicly would require additional authentication and security review.

Value: The operating organization can substitute its own Apertus inference host, retain code locally and audit why a patch was accepted. CPU-only isolated test execution avoids consuming additional model tokens to decide whether a patch passes. The architecture is modular enough to add real language/repository adapters, permissioned tools and independent benchmark datasets.

Next work: Run multiple genuine Apertus v1.5 model trials and preserve unsuccessful attempts; evaluate against licensed real-world issues with an independent hidden holdout; add repository-scale localization; strengthen sandbox isolation and formal provenance; measure end-to-end latency, prompt budget, total cost and failure rates; and integrate the evidence format into the existing Scythe review gate.

Licences and pre-existing intellectual property

New source and build files: **Apache-2.0**. Authored documentation, report and demo script: **CC BY 4.0**. Original dataset: **CDLA-Permissive-2.0**. The pre-existing private Scythe monorepo is *not included*, and no rights to that unrelated code are implied. All newly authored code in this bundle is independently redistributable.

References

1. Xia, C. S., et al. *Agentless: Demystifying LLM-based Software Engineering Agents.* 2024. https://arxiv.org/abs/2407.01489
2. Zhang, Y., et al. *AutoCodeRover: Autonomous Program Improvement.* 2024. https://arxiv.org/abs/2404.05427
3. Jimenez, C. E., et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* 2023. https://arxiv.org/abs/2310.06770
4. Yang, J., et al. *SWE-smith: Scaling Data for Software Engineering Agents.* 2025. https://arxiv.org/abs/2504.21798
5. Hack Apertus organizers. *Official Track 2B project template and submission requirements.* 2026. https://github.com/HackApertus/project-template/tree/main/track_2b
6. Swiss AI Initiative. *Apertus v1.5 8B model card.* https://huggingface.co/swiss-ai/Apertus-v1.5-8B

Documentation and report: Creative Commons Attribution 4.0 International (CC BY 4.0).